

BIG DATA : LAB 2

****Before you start****

Dans cette première partie nous avons écrit dans le fichier source « source .bashrc », afin d'avoir le bon nom du groupe. On utilise les commandes suivantes :

```
```bash
echo "export GROUP=ece_2026_spring_gr01" >> ~/.bashrc
source .bashrc
```
```

On fait en sorte d'initialiser le bon user et le bon groupe :

Export GROUP=ece_2026_spring_gr01

Export USER=l.chloe-ece

On regarde si cela fonctionne avec : echo \$GROUP et echo \$USER

```
[l.chloe-ece@edge-1 ~]$ echo $GROUP
ece_2026_spring_gr01
[l.chloe-ece@edge-1 ~]$ echo $USER
l.chloe-ece
[l.chloe-ece@edge-1 ~]$ |
```

Copy files to HDFS

1. On se place dans le chemin demandé avec la commande :
hdfs dfs -mkdir -p /education/ece_2026_spring_gr01/l.chloe-ece
2. On crée le sous-dossier lab2 :

```
hdfs dfs -mkdir /education/ece_2026_spring_gr01/l.chloe-ece/lab2
```

On vérifie que cela a fonctionné :

```
hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece
```

```
[l.chloe-ece@edge-1 ~]$ hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece
Found 1 items
drwxr-xr-x - l.chloe-ece ece_2026_spring_gr01 0 2026-02-03 08:49 /education/ece_2026_spring_gr01/l.chloe-ece/lab2
[l.chloe-ece@edge-1 ~]$ |
```

3. On crée ensuite le fichier lab 1 :

```
mkdir -p /home/l.chloe-ece/lab1
```

Puis on crée un fichier texte appelé "sentence.txt" et on y ajoute une phrase :

echo "HDFS is a distributed file system used in the Hadoop ecosystem." > /home/l.chloe-ece/lab1/sentence.txt

Vérifions si cela a fonctionné :

```
[l.chloe-ece@edge-1 ~]$ cat /home/l.chloe-ece/lab1/sentence.txt
HDFS is a distributed file system used in the Hadoop ecosystem.
[l.chloe-ece@edge-1 ~]$
```

4. Nous allons maintenant venir ce copier ce fichier dans le dossier lab2 avec la commande suivante :

```
hdfs dfs -put /home/l.chloe-ece/lab1/sentence.txt /education/ece_2026_spring_gr01/l.chloe-ece/lab2/
```

Vérifions que cela a fonctionné : (ici on voit le fichier moby-dick car j'ai réalisé le rapport après avoir finalisé le TP)

```
[l.chloe-ece@edge-1 ~]$ hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/lab2
Found 2 items
drwxr-xr-x - l.chloe-ece ece_2026_spring_gr01 0 2026-02-03 08
:50 /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick
-rw-r--r-- 2 l.chloe-ece ece_2026_spring_gr01 64 2026-02-03 08
:44 /education/ece_2026_spring_gr01/l.chloe-ece/lab2/sentence.txt
[l.chloe-ece@edge-1 ~]$
```

5. Nous allons maintenant afficher les informations sur les blocs HDFS :

```
hdfs fsck /education/ece_2026_spring_gr01/l.chloe-ece/lab2/sentence.txt
```

Nous obtenons :

```
[l.chloe-ece@edge-1 ~]$ hdfs fsck /education/ece_2026_spring_gr01/l.chloe-ece/lab2/sentence.txt
Connecting to namenode via http://hdfs-nn-1.au.adaltas.cloud:50070/fsck?ugi=l.chloe-ece&path=%2Feducation%2Fece_2026_spring_gr01%2Fl.chloe-ece%2Flab2%2Fsentence.txt
FSCK started by l.chloe-ece (auth:KERBEROS_SSL) from /10.0.0.63 for path /education/ece_2026_spring_gr01/l.chloe-ece/lab2/sentence.txt at Tue Feb 03 10:26:12 UTC 2026

Status: HEALTHY
Number of data-nodes: 2
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 64 B
Total files: 1
Total blocks (validated): 1 (avg. block size 64 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 2
Average block replication: 2.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
FSCK ended at Tue Feb 03 10:26:12 UTC 2026 in 1 milliseconds
```

Dans cette partie, nous avons créé un répertoire utilisateur dans l'espace de noms du groupe HDFS, copié un fichier local sur HDFS et utilisé `hdfs fsck` pour examiner la distribution des blocs, le facteur de réplication et les emplacements de stockage.

Submit an application in YARN

1. On commence par lancer l'application MapReduce avec la commande suivante :
yarn jar /usr/hdp/3.1.0.0-78/hadoop-mapreduce/hadoop-mapreduce-examples-3.1.1.3.1.0.0-78.jar pi 6 100000000
2. On attend quelques secondes que cela s'exécute et on obtient
"Estimated value of Pi is 3.14159..."

```
Job Finished in 33.701 seconds
Estimated value of Pi is 3.1415927200000000000000
```

3. On lance maintenant la commande : yarn app -list et on obtient un tableau avec les informations pour toutes les applications :

Application-Name : PiEstimator

State : FINISHED

Final-State : SUCCEEDED

Dans cette partie, nous avons soumis une tâche MapReduce à YARN en utilisant le fichier JAR d'exemples Hadoop pour calculer π par une méthode quasi-Monte Carlo. L'application s'est exécutée avec succès et est apparue dans YARN avec l'état SUCCEEDED.

WordCount example application

1. On va regarder le contenu du dossier en utilisant la commande suivante :
hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/lab2/
on obtient : (le fichier moby-dick n'est pas censé apparaître à cette étape mais j'ai réalisé le rapport après finalisation du TP)

```
[l.chloe-ece@edge-1 ~]$ hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/lab2/
Found 2 items
drwxr-xr-x - l.chloe-ece ece_2026_spring_gr01      0 2026-02-03 08:50 /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick
-rw-r--r-- 2 l.chloe-ece ece_2026_spring_gr01    64 2026-02-03 08:44 /education/ece_2026_spring_gr01/l.chloe-ece/lab2/sentence.txt
```

On voit bien le fichier sentence.txt, que nous allons utiliser comme entrée pour WordCount

2. Nous allons maintenant lancer l'application WordCount avec yarn en utilisant la commande suivante :

```
yarn jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-mapreduce-examples-3.1.1.3.1.0.0-78.jar \
```

```
wordcount /education/$GROUP/$USER/lab2 \
/education/$GROUP/$USER/lab2/output-moby-dick
```

3. Puis nous vérifions le fichier de sortie :

```
hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick
```

```
[l.chloe-ece@edge-1 ~]$ hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick
Found 2 items
-rw-r--r-- 2 l.chloe-ece ece_2026_spring_gr01      0 2026-02-03 08:50 /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick/_SUCCESS
-rw-r--r-- 2 l.chloe-ece ece_2026_spring_gr01    86 2026-02-03 08:50 /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick/part-r-00000
```

On vient ensuite afficher le résultat du WordCount :

```
hdfs dfs -cat /education/ece_2026_spring_gr01/l.chloe-ece/lab2/output-moby-dick/part-r-00000
```

On obtient donc le résultat ci-dessous dans lequel chaque lit correspond à : mot + nombre d'occurrence de ce mot

```
[l.chloe-ece@edge-1
HDFS      1
Hadoop    1
a          1
distributed 1
ecosystem. 1
file      1
in        1
is        1
system    1
the       1
used      1
```

Nous avons exécuté l'exemple WordCount MapReduce sur des fichiers stockés dans HDFS. La tâche a généré avec succès un répertoire de sortie contenant les résultats de fréquence des mots produits par les réducteurs.

Setting up password-less SSH

L'objectif de cette partie est de pouvoir me connecter sur mon ssh sans utiliser le mot de passe. Nous allons utiliser les commandes suivantes, en suivant le site donné dans l'énoncé :

On regarde si on a id_rsa dans le ssh : `ls ~/.ssh`

Si on ne l'a pas, on l'ajoute : `ssh-keygen -t rsa -b 4096`

On vient copier la clé publique sur l'edge node : `ssh-copy-id l.chloe-ece@edge-1.au.adaltas.cloud`

Puis il nous a demandé de remettre le mot de passe.

On va ensuite tenter de se reconnecter afin de vérifier si cela a fonctionné :

```
[l.chloe-ece@edge-1 ~]$ ssh l.chloe-ece@edge-1.au.adaltas.cloud
Last login: Tue Feb  3 10:18:20 2026 from 10.0.0.8
```

On voit bien ici que le mot de passe n'est pas redemandé

Accessing Hadoop Web UIs using Kerberos

1. On vient télécharger l'application MIT Kerberos Ticket Manager
2. On va maintenant créer le fichier krb5.ini :
 - On ouvre l'application Bloc-note du PC en mode administrateur,
 - Puis on ouvre le fichier qui se trouve au chemin suivant
« C:\ProgramData\MIT\Kerberos5\ »
 - On vient y ajouter le contenu suivant :

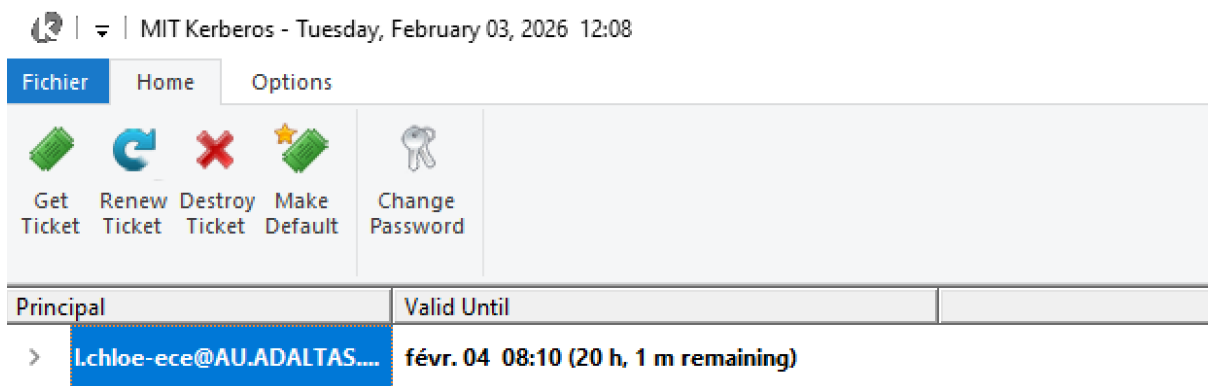
```
[libdefaults]
```

```
default_realm = AU.ADALTAS.CLOUD
dns_lookup_realm = false
dns_lookup_kdc = true
rdns = false
ticket_lifetime = 24h
forwardable = true
udp_preference_limit = 0
```

```
AU.ADALTAS.CLOUD = {
  kdc = ipa1.au.adaltas.cloud:88
  master_kdc = ipa1.au.adaltas.cloud:88
  admin_server = ipa1.au.adaltas.cloud:749
  default_domain = au.adaltas.cloud
}
```

```
[domain_realm]
.au.adaltas.cloud = AU.ADALTAS.CLOUD
au.adaltas.cloud = AU.ADALTAS.CLOUD
ipa1.au.adaltas.cloud = AU.ADALTAS.CLOUD"
```

- On enregistre les modifications
- 3. Création du ticket Kerberos sur l'application MIT, on crée un ticket avec
 - Principal :l.chloe-ece@AU.ADALTAS.CLOUD Password: le même que ssh



- 4. On met ce ticket en défaut en cliquant sur le bouton vert avec l'étoile
- 5. On telecharge l'application firefox et on va venir le configurer :
 - On tape : about:config
 - On va venir modifier les 2 clés suivantes en ajoutant : .au.adaltas.cloud
network.negotiate-auth.trusted-uris et network.negotiate-auth.delegation-uris

| | |
|--|-------------------|
| network.negotiate-auth.delegation-uris | |
| network.negotiate-auth.delegation-uris | .au.adaltas.cloud |

On obtient ce résultat pour les 2 cas.

- 6. Enfin, on force firefox à ne pas utiliser le SSPI, dans les config on cherche donc :
network.auth.use-sspi et on le passe a false

| | |
|-----------------------|-------|
| network.auth.use-sspi | |
| network.auth.use-sspi | false |

- 7. On redémarre firefox et on tape l'URL : <http://yarn-rm-1.au.adaltas.cloud:8088/ui2/#/yarn-apps/apps> , on obtient

| Application ID | Application Type | Application Name | User | State | Queue | Progress | Start Time |
|--------------------------------|------------------|------------------|------------|----------|---------|----------|--------------|
| application_1756207987441_1813 | MAPREDUCE | QuasiMonteCarlo | l.chloe... | Finis... | default | 100% | 2026/02/0... |
| application_1756207987441_1759 | MAPREDUCE | word count | l.chloe... | Finis... | default | 100% | 2026/02/0... |
| application_1756207987441_1756 | MAPREDUCE | QuasiMonteCarlo | l.chloe... | Finis... | default | 100% | 2026/02/0... |

Cette capture d'écran montre l'interface Web du ResourceManager YARN après la configuration de l'authentification Kerberos sur la machine cliente. Plusieurs applications MapReduce soumises par l'utilisateur l.chloe-ece sont visibles, notamment les jobs *QuasiMonteCarlo* (estimation de π) et *WordCount*. Toutes les applications sont dans l'état **FINISHED** avec une progression de **100 %**, ce qui confirme leur exécution correcte. L'interface permet également de visualiser les identifiants des applications, les files d'attente utilisées, les temps de démarrage et l'état d'avancement, illustrant le rôle de YARN dans la gestion et le suivi des tâches distribuées du cluster Hadoop

- Nous allons maintenant lancer l'exemple python une 2eme fois avec la commande :
« yarn jar /usr/hdp/3.1.0.0-78/hadoop-mapreduce/hadoop-mapreduce-examples-3.1.1.3.1.0.0-78.jar pi 6 100000000 » et on voit dans l'application UI Yarn, que cela vient d'apparaître

| Application Name | User | State | Queue | Progress | Start Time | Elapsed Time | Finished Time |
|------------------|------------|----------|---------|----------|--------------------|--------------|--------------------|
| QuasiMonteCarlo | l.chloe... | Accepted | default | 0% | 2026/02/03 12:1... | 2s 901ms | 1970/01/01 01:0... |

Nous allons relancer 3 fois la même commande et analyser les différences :

| Application Attempt | Application Attempt ID | Started Time | Elapsed Time | AM Container ID | AM Node ID | AM Node Web UI | Log |
|---------------------------------------|---------------------------------------|---------------------|--------------|----------------------------------|----------------------------|----------------------------|-----|
| application_1756207987441_1813_000001 | application_1756207987441_1813_000001 | 2026/02/03 12:20:59 | 9 Secs | container_186_1756207987441_1... | win-2.acadallux.cloud:5456 | win-2.acadallux.cloud:5456 | Log |
| application_1756207987441_1813_000002 | application_1756207987441_1813_000002 | 2026/02/03 12:22:25 | 19 Secs | container_186_1756207987441_1... | win-2.acadallux.cloud:5456 | win-2.acadallux.cloud:5456 | Log |
| application_1756207987441_1813_000003 | application_1756207987441_1813_000003 | 2026/02/03 12:22:25 | 31 Secs | container_186_1756207987441_1... | win-2.acadallux.cloud:5456 | win-2.acadallux.cloud:5456 | Log |

L'application Pi MapReduce a été exécutée trois fois et surveillée à l'aide de l'interface web YARN ResourceManager. Bien que l'ApplicationMaster ait été systématiquement planifié sur le même nœud, le temps d'exécution a varié considérablement d'une exécution à l'autre (31 s, 9 s et 19 s). Ces variations illustrent le comportement de planification dynamique de YARN, l'allocation des ressources dépendant de la charge du cluster et de la disponibilité des ressources au moment de la soumission. Ceci démontre comment YARN gère et équilibre les ressources au sein du cluster.